UNDEF

FADENA
Facultad de Defensa Nacional

Facultad de Defensa Nacional

Carrera: Lic. en Ciberdefensa

Materia
Sistemas Operativos I

Unidad 2 / Semana 5

Año: 2025

Contenidos

Sistemas operativos: clasificación, tipos y estructura. Interfaz de usuario. Evolución histórica de los sistemas operativos.

Índice

1. Introducción 3
1.1 Los sistemas operativos 3
1.2 Relación del sistema operativo con el hardware y el usuario 3
1.2.1 Administración de recursos en sistemas operativos 4
1.2.1.1 Multiplexación en el tiempo 4
Analogía para comprender la multiplexación 4
1.3 Ejemplos 4
1.4 Objetivos de un sistema operativo 5
1.5 Funciones del sistema operativo 5
1.6 Interfaces de usuario 6
2. Clasificación de los sistemas operativos 6
2.1 Según su estructura 7
2.1.1 Monolíticos 7
2.1.2 Sistemas Jerárquicos o de capas 7
2.1.3 Microkernels 8
2.1.4 Cliente / Servidor 9
2.1.4 Máquinas virtuales 9
2.1.5 Exokernel 10
2.2 Según los servicios que ofrecen 10
2.2.1 Monousuarios 11
2.2.2 Multiusuarios 11
2.2.3 Monotareas 11
2.2.4 Multitareas 11
2.2.5 Uniproceso 12
2.2.6 Multiproceso 12
2.2.6.1 Simétrico 12
2.2.6.2 Asimétrico 12
2.3 Según la forma en que entrega sus servicios 12
2.3.1 Sistemas operativos de red 12
2.3.2 Sistemas operativos distribuídos 13
2.4 Según la interacción con el sistema 13
2.4.1 Sistemas Batch o por Lotes 13
2.4.2 Sistemas Interactivos o time sharing 14
2.4.3 Sistemas operativos de tiempo real (RTOS) 15
2.4.3.1 RTOS de tiempo real duro 15
2.4.3.2 RTOS de tiempo real suave 16
2.5 Tipos de Sistemas Operativos 16
2.5.1 Mainframe 16
2.5.2 Servidores 16
2.5.3 Múltiples procesadores / paralelo 16
2.5.4 Computadoras personales 16
2.5.5 Computadoras de bolsillo 17
2.5.6 Embebidos o integrados 17
2.5.7 Sensores 17
2.5.8 Sistemas operativos de tiempo real 17

2.5.9 Sistemas operativos de tarjetas inteligentes 17
3. Evolución Histórica de los Sistemas Operativos 17
4. Síntesis del módulo 18

1. Introducción

Comenzaremos definiendo qué es un sistema operativo, así como sus objetivos y funciones. También exploraremos las diferentes clasificaciones, su evolución histórica, relacionándola directamente con los avances en el hardware que hicieron posible la creación de sistemas operativos con capacidades cada vez más avanzadas. Este enfoque nos permitirá no solo comprender por qué surgieron ciertas características y patrones de diseño, sino también cómo los sistemas existentes se han adaptado a entornos en constante cambio.

Se destacarán las características más relevantes de los sistemas operativos modernos y se presentarán conceptos fundamentales, con el objetivo de clarificar las relaciones entre las distintas funciones de un sistema operativo.

Además, introduciremos conceptos clave para entender cómo funcionan los sistemas operativos, tales como monoprogramación, multiprogramación, multiprocesamiento y procesamiento distribuido.

Finalmente, cerraremos esta primera clase describiendo los principales logros alcanzados por los sistemas operativos modernos.

1.1 Los sistemas operativos

Un sistema operativo está constituido por un conjunto de programas que actúan como intermediarios entre el usuario y el hardware del sistema informático. Además, controla la ejecución de los programas de aplicación proporcionando un entorno para su ejecución. Estos programas utilizan las facilidades que brinda el sistema operativo para acceder a recursos del sistema informático como el procesador, la memoria disponible, archivos y dispositivos de E/S. De esta manera, el sistema operativo constituye la base sobre la que se escriben los programas de aplicación.

En otras palabras, un sistema operativo es el conjunto de programas encargados de administrar los recursos de un sistema de cómputo, tales como la CPU, memoria y los dispositivos de Entrada y Salida (E/S) a fin de satisfacer los requerimientos de los usuarios.

1.2 Relación del sistema operativo con el hardware y el usuario

El sistema operativo se relaciona directamente con el hardware del sistema informático y sirve de plataforma para la ejecución de los programas de aplicación. En la figura se muestra el nivel que ocupa en un sistema informático:



En el sistema operativo se encuentra una capa de software que permite relacionarse directamente con el hardware (capa inferior o kernel) y, por otra parte, otra que permite instalar y ejecutar aplicaciones (capa superior).

El **kernel** o **núcleo** del sistema operativo es el componente fundamental y se encarga de gestionar (compartir) los principales recursos del sistema como procesador, memoria principal, etc.

1.2.1 Administración de recursos en sistemas operativos

La administración de recursos en un sistema operativo se basa en la multiplexación, que es una técnica para compartir recursos de manera eficiente. Esta multiplexación puede ocurrir de dos maneras: en el tiempo y en el espacio.

1.2.1.1 Multiplexación en el tiempo

Cuando un recurso es indivisible y no puede ser compartido simultáneamente, se multiplexa en el tiempo, permitiendo que diferentes programas o usuarios lo utilicen por turnos.

Por ejemplo, en un sistema con una sola CPU y múltiples programas en ejecución, el sistema operativo asigna la CPU a un programa durante un período determinado. Luego, cambia la asignación a otro programa y repite el proceso.

1.2.1.2 Multiplexación en el espacio

Por el contrario, cuando un recurso es divisible o fraccionable, se multiplexa en el espacio, lo que significa que varios programas o usuarios pueden utilizarlo simultáneamente, pero asignándose porciones independientes del recurso.

A diferencia de la multiplexación en el tiempo, aquí no hay turnos, sino una distribución física del recurso entre los diferentes programas. Ejemplo: Multiplexación de la memoria RAM o del almacenamiento en disco.

1.2.1.3 Analogía para comprender la multiplexación

Para visualizar mejor este concepto, podemos utilizar dos ejemplos cotidianos: una bicicleta y una pizza.

- Bicicleta (Multiplexación en el tiempo): No podemos dividir una bicicleta a la mitad sin inutilizarla, por lo que la mejor solución es asignar intervalos de tiempo de uso entre los usuarios.

- Pizza (Multiplexación en el espacio): Al ser divisible, basta con cortarla en porciones y distribuirlas entre varias personas, permitiendo el consumo simultáneo sin necesidad de esperar turnos.

1.3 Ejemplos

Son algunos ejemplos de sistemas operativos:

- CP/M
- Unix
- MS-DOS
- OS/2
- GNU/Linux
- FreeBSD
- Microsoft Windows
- Macintosh OS
- Android

1.4 Objetivos de un sistema operativo

Según Stallings[1], puede considerarse que un sistema operativo tiene tres objetivos:

A. **Comodidad**: debe permitir que un sistema informático sea más fácil de utilizar.
B. **Eficiencia**: debe permitir una administración eficiente de los recursos del sistema informático.
C. **Capacidad de evolución**: debe permitir su evolución en el tiempo debido a actualizaciones del hardware, nuevos tipos de hardware o la necesidad de dar respuesta a las necesidades de nuevos servicios por parte de los usuarios.

1.5 Funciones del sistema operativo

Algunas de las funciones básicas son:

- **Administrar los recursos**. Un sistema computacional dispone de múltiples recursos, como memoria, almacenamiento y tiempo de procesamiento, entre otros. Los procesos que se ejecutan dentro de este sistema compiten por dichos recursos, lo que requiere una gestión eficiente para garantizar un funcionamiento óptimo. El sistema operativo desempeña un papel fundamental en esta tarea, implementando políticas que **asignan los recursos** de manera efectiva y en función de las necesidades específicas del sistema.
- **Proporcionar una interfaz con el usuario**. El sistema operativo facilita la interacción entre el usuario y el hardware del dispositivo, ofreciendo

[1] Stallings, William (2005), Sistemas operativos. Aspectos internos y principios de diseño, Quinta edición, pág. 54.

entornos gráficos o de línea de comandos que permiten ejecutar programas, administrar archivos y configurar el sistema de manera intuitiva.

- **Gestionar los procesos**, para que los programas puedan ejecutarse de manera correcta.
- **Administrar y controlar los dispositivos externos**, por ejemplo: micrófonos, altavoces, impresoras o el monitor.
- **Gestionar el sistema de archivos**.
- **Garantizar la seguridad del sistema informático**, impidiendo el acceso a ciertos archivos o programas para el correcto funcionamiento del equipo.
- **Detectar errores y mantener la operatividad**.

Es importante comprender que las funciones del sistema operativo están enfocadas en regular y gestionar la ejecución de cada aplicación, programa y recurso del sistema informático.

1.6 Interfaces de usuario

La **interfaz de usuario**, IU (del inglés User Interface, UI) es el medio por el cual las personas nos comunicamos con las máquinas. Esta interacción está mediada por controles y comandos, nos permite utilizar dispositivos como smartphones, computadoras, cajeros automáticos o vehículos.

Según la forma de interactuar del usuario existen diferentes tipos de interfaces de usuario. Entre ellas:

- **Interfaz de línea de comandos** (del inglés Command-Line Interface, **CLI**): Este tipo de interfaz sólo permite el uso de texto. El usuario deberá escribir comandos que interactúen con el sistema operativo o software de aplicación de un equipo.
- **Interfaces gráficas de usuario** (del inglés Graphic User Interface, **GUI**): Es una evolución de la línea de comandos que permite al usuario comunicarse con la computadora o dispositivo de forma rápida e intuitiva, normalmente con el ratón a través de ventanas y botones, representando gráficamente todos los elementos de control y medida necesarios para la comunicación usuario/computadora.



Actividad S5-A1

Realice la actividad identificada como *S5-A1* en la sección de actividades del campus. Verá ejemplos históricos de interfaces de usuario de sistemas operativos y preguntas guía para investigar y profundizar el tema.

2. Clasificación de los sistemas operativos

Los sistemas operativos pueden clasificarse usando diferentes criterios, según su estructura interna, los servicios que ofrecen, la forma en que proporcionan dichos servicios y la forma de interacción con estos sistemas entre otros.

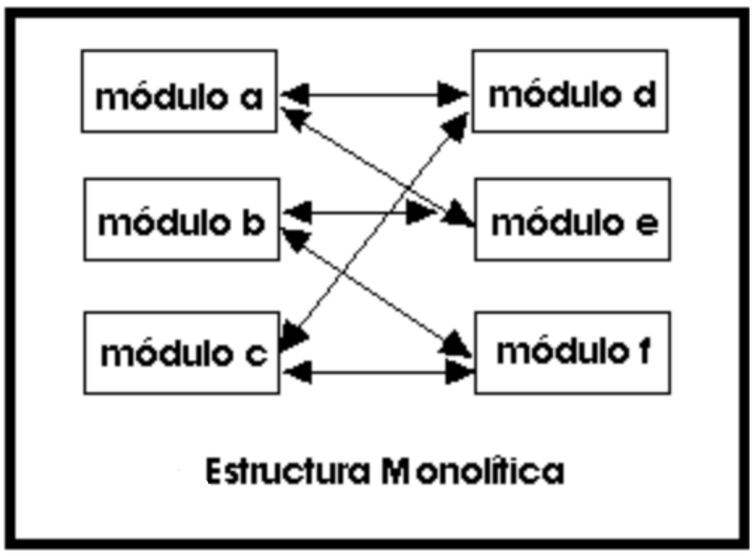
Esta clasificación permite entender su arquitectura y funcionalidad en distintos entornos computacionales.

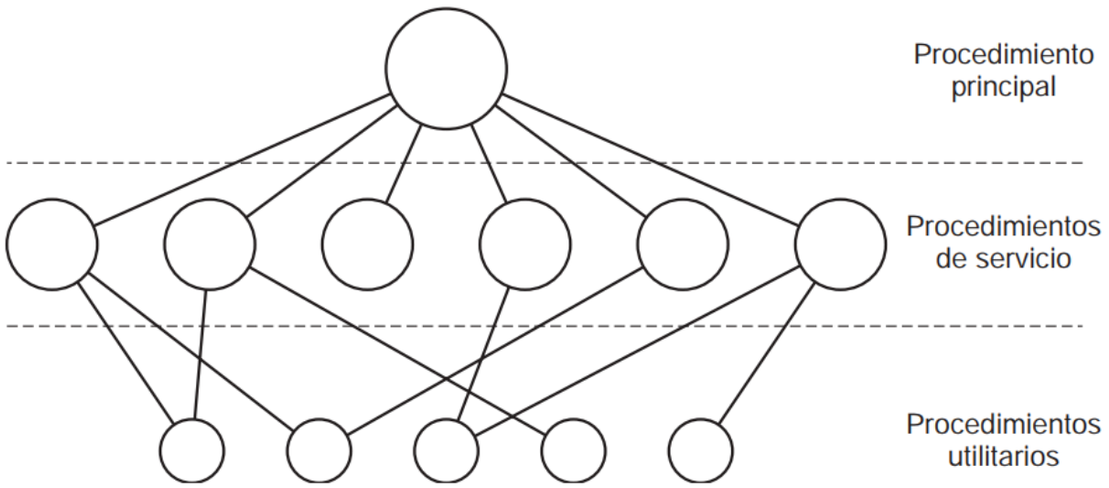
2.1 Según su estructura

2.1.1 Monolíticos

Los sistemas operativos monolíticos están compuestos por un único programa con múltiples rutinas interconectadas. Aunque ofrecen eficiencia en ejecución, presentan problemas de modularidad y seguridad debido a la falta de aislamiento entre componentes. Sin embargo se pueden organizar en su desarrollo de forma tal que tenga una estructura básica que facilite su comprensión y mantenimiento.

Ejemplos clásicos incluyen MS-DOS y versiones tempranas de Unix, donde el núcleo del sistema operativo contenía todas las funciones esenciales sin separación clara entre módulos.





Un modelo de estructuración simple para un sistema monolítico[2]

[2] Tanenbaum, Andrew S. (2009). Sistemas Operativos Modernos. 3era edición. Pearson Educación, México

2.1.2 Sistemas Jerárquicos o de capas

La evolución de los sistemas operativos llevó a la organización en niveles, donde cada capa tiene una función específica.

El primer ejemplo de esta estructura fue el sistema THE (1968, W.Dijkstra) construido en Holanda en el *Technische Hogeschool Eindhoven*, de ahí su nombre.

Otros ejemplos incluyen Multics y Unix, que implementan una arquitectura multicapa para mejorar la modularidad y seguridad.

Los sistemas operativos modernos, como Linux, han adoptado una estructura modular que permite la carga dinámica de componentes, mejorando la estabilidad y flexibilidad.

Capa	Función
5	El operador
4	Programas de usuario
3	Administración de la entrada/salida
2	Comunicación operador-proceso
1	Administración de memoria y tambor
0	Asignación del procesador y multiprogramación

Estructura del sistema operativo THE[3]

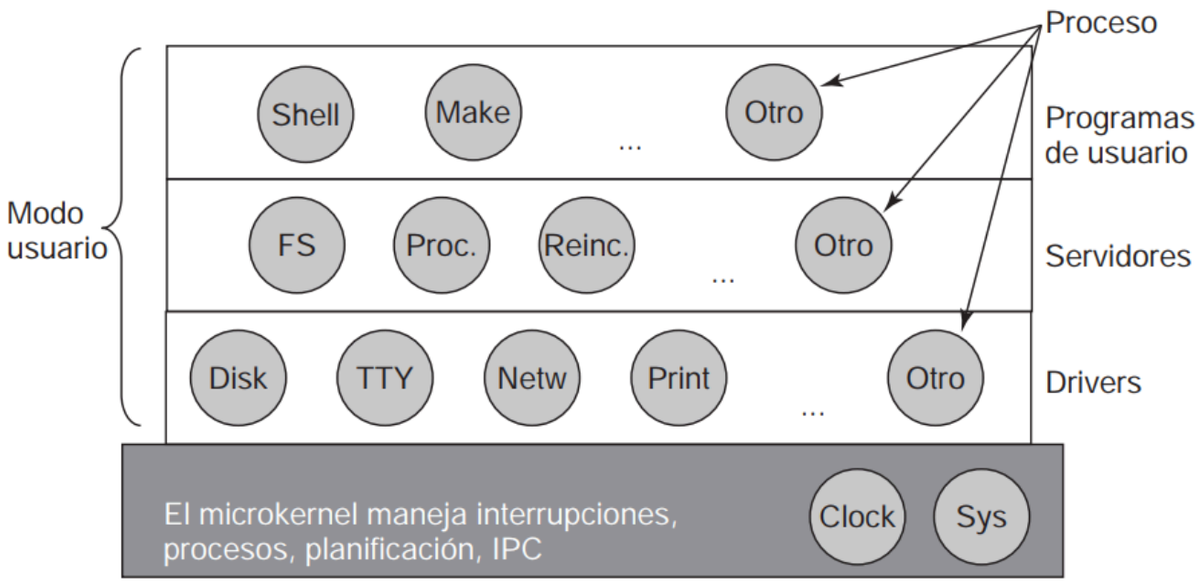
2.1.3 Microkernels

Los microkernels han ganado relevancia debido a su diseño modular, donde sólo las funciones esenciales residen en el núcleo, mientras que los servicios adicionales se ejecutan en espacio de usuario. Esto genera una mayor estabilidad al sistema, limitando que una falla menor, por ejemplo en un driver, no detenga al sistema completo.

Ejemplos clásicos de esta estructura son Minix 3, Symbian y más modernos incluyen seL4, Fuchsia y QNX, que ofrecen mayor seguridad y estabilidad.

Los microkernels modernos han mejorado la comunicación entre procesos mediante técnicas avanzadas de Inter-Process Communication (IPC), reduciendo la latencia y mejorando la eficiencia en sistemas embebidos y de tiempo real.

[3] Tanenbaum, Andrew S. (2009). Sistemas Operativos Modernos. 3era edición. Pearson Educación, México

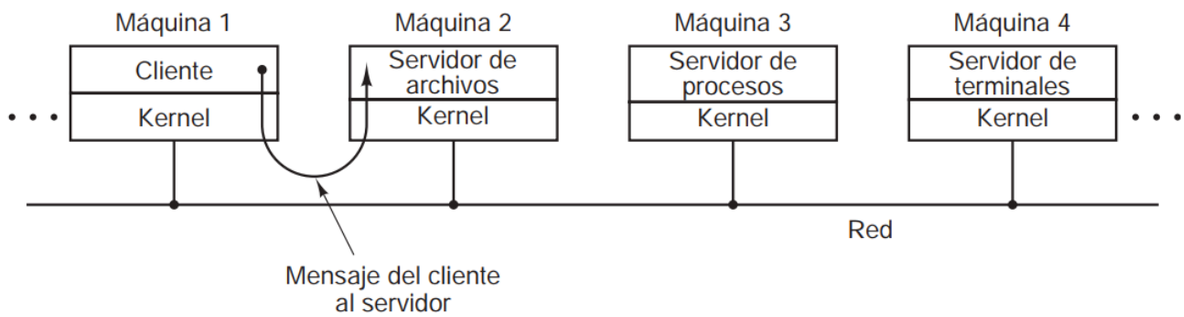


Estructura del sistema MINIX 3[4]

2.1.4 Cliente / Servidor

El modelo cliente / servidor es una ligera variación del concepto de microkernel diferenciando dos clases de procesos, los *servidores,* que proveen cierto servicio y los *clientes,* que los consumen o utilizan. La comunicación entre ambos es a través de mensajes.

Una generalización de esta idea es que los clientes y los servidores pudieran estar en distintas computadoras, conectadas a través de una red local o internet, como es el caso del acceso de una computadora personal a un sitio en Internet.



Modelo cliente / servidor sobre una red de computadoras[5]

2.1.4 Máquinas virtuales

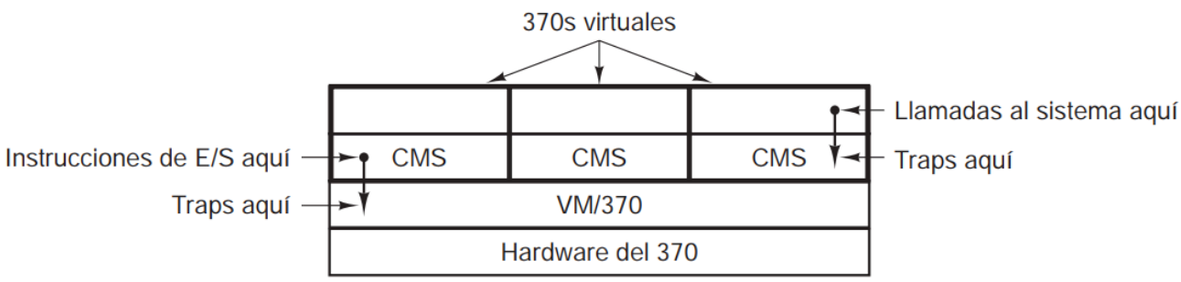
Los sistemas operativos de máquina virtual permiten la ejecución simultánea de múltiples instancias de sistemas operativos sobre un mismo hardware físico, proporcionando una abstracción completa del entorno subyacente. Estas instancias virtualizadas pueden replicar el sistema operativo anfitrión o ejecutar sistemas operativos distintos, dependiendo de la configuración y los requerimientos del usuario.

[4] Tanenbaum, Andrew S. (2009). Sistemas Operativos Modernos. 3era edición. Pearson Educación, México

[5] Tanenbaum, Andrew S. (2009). Sistemas Operativos Modernos. 3era edición. Pearson Educación, México

En este contexto, cada máquina virtual (VM) opera de manera independiente, con su propio sistema de gestión de recursos, procesador virtual, memoria asignada y almacenamiento virtualizado. Para el software que se ejecuta dentro de cada máquina virtual, el entorno es indistinguible de una máquina física real, lo que permite la ejecución de aplicaciones sin modificaciones y con total compatibilidad.

Esta tecnología no es nueva, IBM ya la usaba hace más de 40 años en su modelo VM/370 en 1979. El corazón del sistema era su *monitor de máquina virtual*, quien proporcionaba las máquinas virtuales, aunque en esa época todas eran réplicas del hardware real.



Estructura de la IBM VM/370 con CMS[6]

En los últimos años se produjo un *redescubrimiento* de la tecnología de virtualización, donde empresas como Sun Microsystems y HP incluyeron a esta tecnología dentro de la línea de sus productos de servidores.

Luego, la tecnología de virtualización ha avanzado significativamente, hasta el punto en que cualquier usuario puede crear y gestionar máquinas virtuales en su computadora personal con facilidad. Este desarrollo ha facilitado el acceso a entornos virtualizados, permitiendo la ejecución de múltiples sistemas operativos sin necesidad de hardware adicional dedicado.

Más recientemente, la virtualización ha evolucionado hacia una arquitectura más ligera con el uso de contenedores, como Docker y Kubernetes. Estas tecnologías permiten la ejecución de aplicaciones en entornos completamente aislados, sin necesidad de virtualizar un sistema operativo completo, optimizando el uso de recursos y mejorando la escalabilidad en entornos de desarrollo y producción.

2.1.5 Exokernel

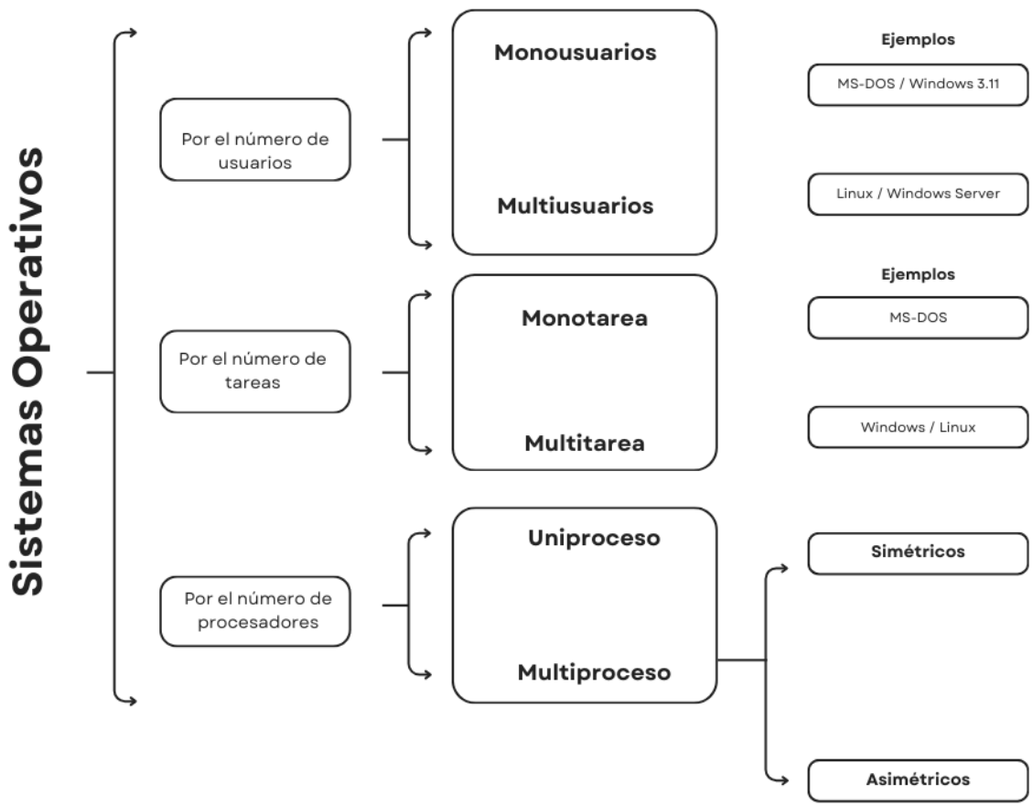
Un exokernel es un tipo de arquitectura de sistema operativo que minimiza las abstracciones impuestas por el núcleo del SO, permitiendo que las aplicaciones accedan directamente al hardware con el menor nivel de intervención posible, a diferencia de las máquinas virtuales que proporcionan servicios como gestión de memoria o sistemas de archivos, el exokernel simplemente multiplexa los recursos físicos y deja que las aplicaciones o bibliotecas definan sus propias abstracciones.

2.2 Según los servicios que ofrecen

Esta clasificación es la más comúnmente usada y conocida desde el punto de vista del usuario final.

[6] Tanenbaum, Andrew S. (2009). Sistemas Operativos Modernos. 3era edición. Pearson Educación, México

Se puede comprender fácilmente con el cuadro sinóptico que a continuación se muestra:



2.2.1 Monousuarios

Los sistemas operativos monousuarios son aquéllos que soportan a un usuario a la vez, sin importar el número de procesadores que tenga la computadora o el número de procesos o tareas que el usuario pueda ejecutar en un mismo instante de tiempo. Las computadoras personales típicamente se han clasificado en este renglón.

2.2.2 Multiusuarios

Los sistemas operativos multiusuarios son capaces de dar servicio a más de un usuario a la vez, ya sea por medio de varias terminales conectadas a la computadora central o por medio de sesiones remotas en una red de comunicaciones. No importa el número de procesadores en la máquina ni el número de procesos que cada usuario puede ejecutar simultáneamente.

2.2.3 Monotareas

Los sistemas monotarea son aquellos que sólo permiten una tarea a la vez por usuario. Puede darse el caso de un sistema multiusuario y monotarea, en el cual se admiten varios usuarios al mismo tiempo pero cada uno de ellos puede estar haciendo solo una tarea a la vez.

2.2.4 Multitareas

Un sistema operativo multitarea es aquél que le permite al usuario estar realizando varias labores al mismo tiempo. Por ejemplo, puede estar editando un documento mientras actualiza una planilla de cálculo y a la vez que está recibiendo correo electrónico en un proceso en background. Es común encontrar

en ellos interfaces gráficas orientadas al uso de menús, mouse o pantalla táctil para facilitar el rápido intercambio entre las tareas para el usuario, mejorando su productividad.

2.2.5 Uniproceso

Un sistema operativo uniproceso es aquél que es capaz de manejar solamente un *procesador* de la computadora, de manera que si la computadora tuviese más de uno no podría utilizarlo de ninguna forma. El ejemplo más típico de este tipo de sistemas es el MS-DOS y versiones antiguas de MacOS.

2.2.6 Multiproceso

Es un sistema operativo capaz de gestionar múltiples procesadores, distribuyendo la carga de trabajo entre ellos para mejorar el rendimiento del sistema.

Existen aplicaciones que fueron hechas para correr en sistemas monoproceso que no toman ninguna ventaja a menos que el sistema operativo o el compilador detecte secciones de código paralelizable, los cuales son ejecutados al mismo tiempo en procesadores diferentes.

Por otro lado, el programador puede tomar en cuenta esta ventaja y aprovecharla incorporando en sus programas esta capacidad, que debe estar disponible previamente en el lenguaje de programación utilizado, ejemplo C, C++, Python etc.

Generalmente estos sistemas trabajan de dos formas: simétrica o asimétrica.

2.2.6.1 Simétrico

Cuando el sistema trabaja de manera simétrica, los procesos o partes de ellos (threads) son enviados indistintamente a cualquiera de los procesadores disponibles, teniendo, teóricamente, una mejor distribución y equilibrio en la carga de trabajo bajo este esquema.

2.2.6.2 Asimétrico

Cuando se trabaja de manera asimétrica, el sistema operativo selecciona a uno de los procesadores el cual jugará el papel de procesador maestro y servirá como pivote para distribuir la carga a los demás procesadores, que reciben el nombre de esclavos (master / slave).

2.3 Según la forma en que entrega sus servicios

2.3.1 Sistemas operativos de red

Los sistemas operativos de red están diseñados para facilitar la comunicación y la interacción entre los sistemas operativos de múltiples computadoras independientes, permitiendo la transferencia de información, el intercambio de archivos, la ejecución de comandos remotos y una amplia variedad de operaciones distribuidas. Estos sistemas son fundamentales en entornos colaborativos y empresariales, donde varios usuarios requieren acceso simultáneo a recursos compartidos.

Actualmente, los sistemas operativos de red cuentan con herramientas y protocolos avanzados como SSH, FTP, NFS y SMB entre otros, que simplifican la gestión de recursos compartidos y mejoran la seguridad en la comunicación entre estos dispositivos.

Ejemplos: windows server, linux, Unix, Novell netware (fue uno de los primeros sistemas operativos de red, muy popular en los años 90, ahora en desuso).

2.3.2 Sistemas operativos distribuídos

Los sistemas operativos distribuidos son aquellos que gestionan múltiples computadoras interconectadas, permitiendo que trabajen como una única entidad lógica. Optimizan el uso de los recursos compartidos, mejoran la tolerancia a fallos y facilitan la ejecución de procesos en paralelo, facilitando el acceso a recursos remotos como si fueran locales en forma transparente.

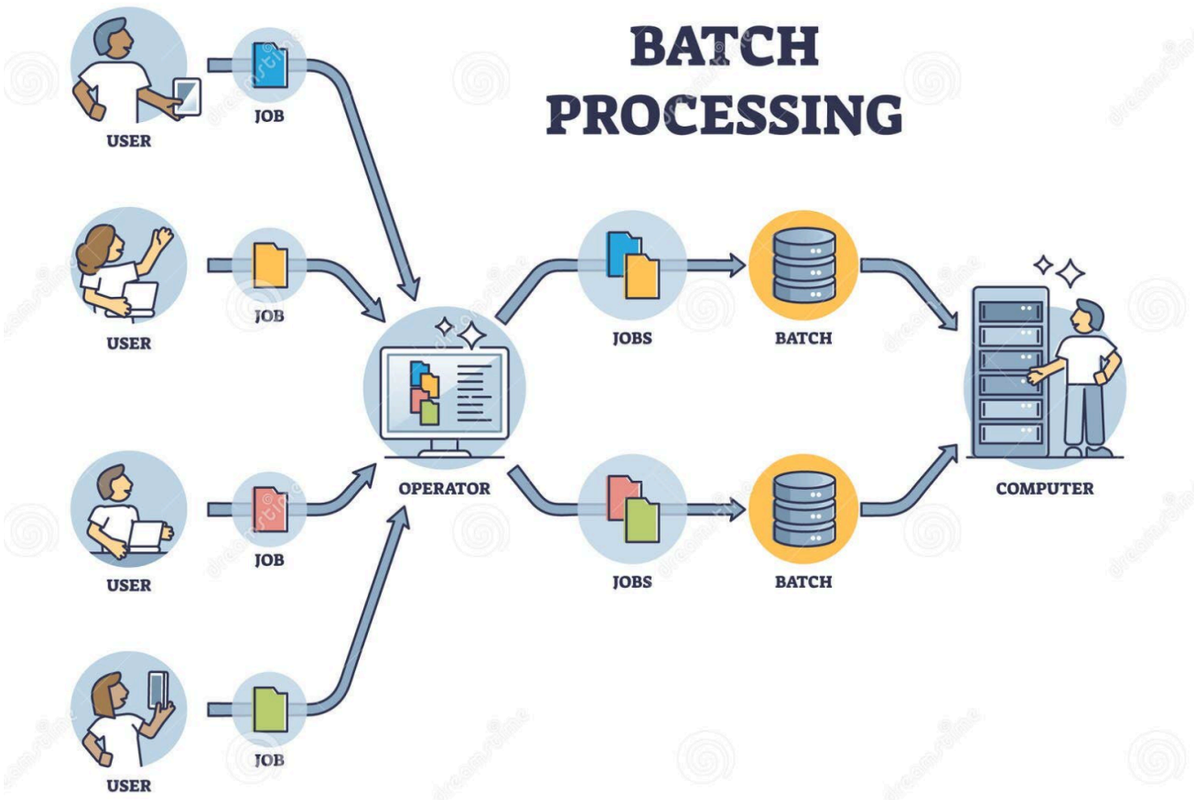
Ejemplos: Google Fuschia, Amoeba, Plan 9.

2.4 Según la interacción con el sistema

2.4.1 Sistemas Batch o por Lotes

Los sistemas batch o por lotes en su modalidad exclusiva por lotes, fueron de los primeros en ser utilizados en la historia. En estos sistemas el procesamiento de los trabajos se ejecuta de manera secuencial, *sin intervención directa del usuario*. Se utilizan principalmente para procesar grandes volúmenes de datos de manera automatizada, optimizando el uso de recursos computacionales.
Los grandes sistemas actuales aún conservan esa modalidad (ya no exclusiva) de trabajo para procesar grandes volúmenes de datos, normalmente en horarios nocturnos.

Ejemplo: IBM OS/360 de IBM, VAX/VMS de Digital Equipment Corp. en su modalidad batch, AOS/VS de Data General en modalidad batch.

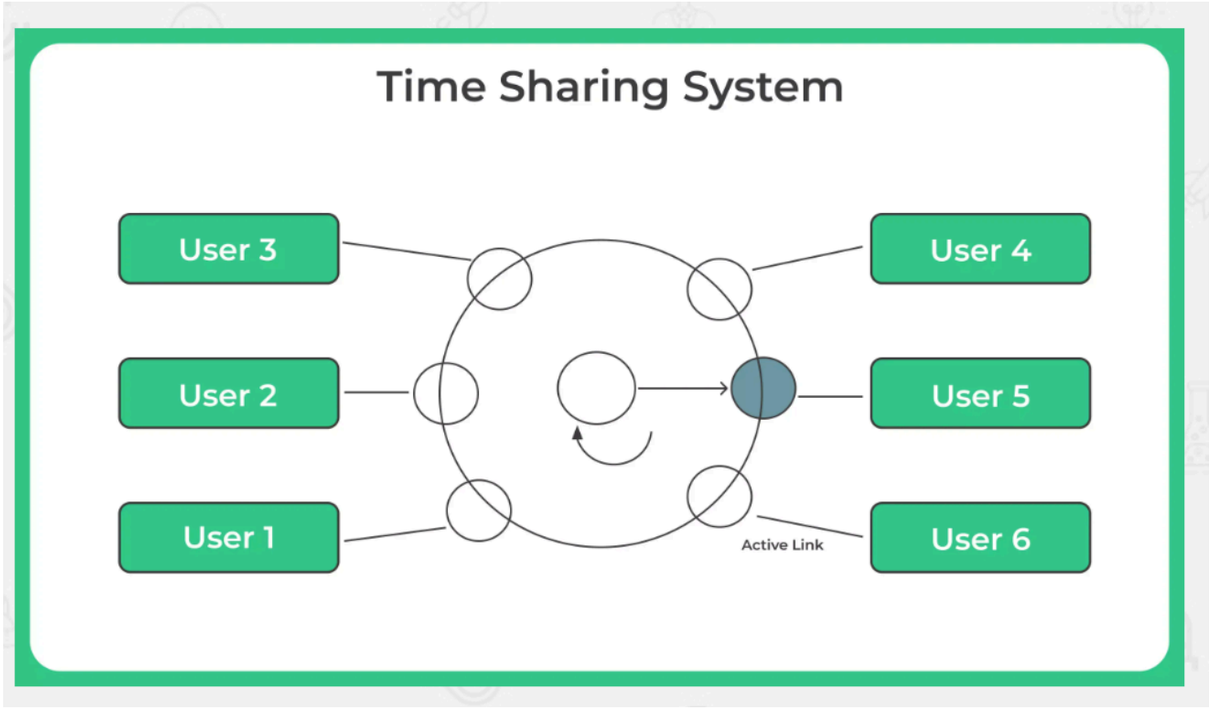


2.4.2 Sistemas Interactivos o time sharing

Con la llegada de los monitores y las terminales, surgieron los sistemas operativos interactivos y de tiempo compartido (time-sharing). Estos sistemas fueron diseñados para permitir que múltiples usuarios o procesos accedan simultáneamente a los recursos de una computadora de manera eficiente, maximizando el uso del procesador y la capacidad del sistema.

El modelo de conexión se basa en la interacción de varios usuarios mediante sus terminales, las cuales están conectadas a una computadora central que gestiona las solicitudes y distribuye el tiempo de procesamiento equitativamente. Este esquema permitió una mejor utilización de los recursos computacionales, dando lugar a entornos multiusuario en los que cada usuario percibe que tiene control exclusivo sobre el sistema, aunque en realidad se comparte entre varios procesos.

Ejemplos: Unix, Linux, Solaris, IBM z/OS, AOS/VS de Data General en modalidad interactiva, Bull GCOS7



2.4.3 Sistemas operativos de tiempo real (RTOS)

Los sistemas operativos de propósito general (GPOS, General-Purpose Operating System) pueden operar en modalidad batch (por lotes) o de forma interactiva, cuando permiten la interacción directa con el usuario. Sin embargo, su diseño no es adecuado para la gestión de procesos críticos, ni para la interacción eficiente con máquinas y sensores, como ocurre en el control de procesos industriales.

En estos entornos, la ejecución de tareas no depende de un usuario, sino de dispositivos y sistemas que se comunican a velocidades y niveles de precisión mucho mayores. La respuesta del sistema debe generarse dentro de un tiempo predefinido, con mínimas variaciones, ya que cualquier retraso o inconsistencia puede comprometer la operación y la seguridad del proceso.

Para abordar estos desafíos, se desarrollaron los sistemas operativos de tiempo real (RTOS, Real-Time Operating System), diseñados específicamente para garantizar tiempos de respuesta predecibles, permitiendo la ejecución precisa de tareas en automatización industrial, robótica, telecomunicaciones y otros sistemas donde la determinación temporal es crítica.

Existen 2 categorías:

2.4.3.1 RTOS de tiempo real duro

Son sistemas operativos diseñados para garantizar que todas las tareas se ejecuten dentro de un tiempo límite estricto, sin excepciones. En estos sistemas, cualquier retraso en la ejecución de una tarea se considera un fallo crítico, ya que pueden estar involucrados en aplicaciones donde la seguridad y la precisión son fundamentales.

Ejemplo: Integrity, QNX, RTEMS usados en aeronautica, aplicaciones espaciales y militares

2.4.3.2 RTOS de tiempo real suave

Estos sistemas priorizan la ejecución de tareas dentro de un tiempo determinado, pero se admite cierta flexibilidad en los plazos. A diferencia de los RTOS de tiempo real duro, donde cualquier retraso es inaceptable, en los RTOS suaves los tiempos de respuesta pueden variar ligeramente sin afectar de manera crítica el funcionamiento del sistema.

Ejemplo: FreeRTOS, QNX, TinyOS (usado en IoT).

2.5 Tipos de Sistemas Operativos

En su libro *Sistemas Operativos Modernos*, el autor Andrew Tanenbaum propone una clasificación adicional de los sistemas operativos, tomando como criterio el ambiente en el que se ejecutan. Esta categorización permite analizar los diferentes tipos de sistemas según su aplicación y entorno operativo, facilitando una mejor comprensión de sus características y funciones.

2.5.1 Mainframe

Un mainframe es una computadora de alto rendimiento y gran capacidad de procesamiento y, sus sistemas operativos, están diseñados para manejar grandes volúmenes de datos y múltiples procesos simultáneamente, principalmente en centros de datos corporativos y/o gubernamentales.

2.5.2 Servidores

En el siguiente nivel hacia abajo se encuentran los sistemas operativos de servidores. Se ejecutan en servidores, que son computadoras de gran capacidad de procesamiento pero con otros objetivos diferentes de los mainframes. Dan servicio a varios usuarios a la vez a través de una red y les permiten compartir los recursos de hardware y de software. Los servidores pueden proporcionar diversos servicios como: de impresión, de archivos o Web.

2.5.3 Múltiples procesadores / paralelo

El uso de múltiples CPU en un solo sistema se ha vuelto común para aumentar la potencia de cómputo, formando computadoras en paralelo, multicomputadoras o multiprocesadores. Estos requieren sistemas operativos especiales, generalmente basados en versiones modificadas de sistemas operativos de servidores, con optimizaciones para comunicación, conectividad y procesamiento en paralelo..

La llegada de los chips multinúcleo ha hecho que incluso los sistemas operativos de computadoras personales deban adaptarse a la ejecución en multiprocesadores. Aunque el conocimiento sobre estos sistemas está bien desarrollado, el desafío sigue siendo optimizar las aplicaciones para aprovechar al máximo su capacidad. Sistemas operativos como Windows y Linux ya están diseñados para operar en entornos multiprocesador.

2.5.4 Computadoras personales

Los sistemas operativos de computadoras personales están diseñados para proporcionar soporte eficiente a un solo usuario, permitiendo la ejecución de múltiples programas simultáneamente mediante multiprogramación. Son ampliamente utilizados en tareas como procesamiento de texto, hojas de cálculo y acceso a Internet y son los que utilizamos a diario.

2.5.5 Computadoras de bolsillo

En este segmento teníamos los dispositivos conocidos como PDA (Personal Digital Assistant) que se fusionaron con los teléfonos celulares y dieron paso a los smartphones. Una característica distintiva es la interfaz que es gráfica y con una pantalla táctil.

2.5.6 Embebidos o integrados

En este caso los sistemas operativos son adaptados o diseñados especialmente para funcionar en algún dispositivo, por ejemplo un cajero automático.

2.5.7 Sensores

El uso de sensores se hace cada vez más más frecuente y se usan para distintos fines. Estos sensores son pequeñas computadoras que tienen CPU, RAM y conectividad normalmente inalámbrica para comunicarse y enviar los datos que recolectan.

2.5.8 Sistemas operativos de tiempo real

Estos tipos de sistemas ya fueron mencionados anteriormente, no ahondaremos en este tema en esta oportunidad.

2.5.9 Sistemas operativos de tarjetas inteligentes

Los sistemas operativos de tarjetas inteligentes son un tipo de software específico, diseñado para gestionar la seguridad, el almacenamiento y la ejecución de aplicaciones en tarjetas inteligentes. Estas tarjetas contienen un chip de CPU, memoria y protocolos de comunicación, lo que les permite realizar funciones como autenticación, pagos electrónicos y almacenamiento seguro de datos.

3. Evolución Histórica de los Sistemas Operativos

La historia de los sistemas operativos está estrechamente ligada a la evolución de la informática, desde las primeras computadoras hasta los sistemas modernos de alto rendimiento. En sus inicios, las computadoras no contaban con un sistema operativo como lo conocemos hoy; cada programa debía comunicarse directamente con el hardware, lo que hacía el proceso lento y complicado.



Actividad S5-A2

Realice la actividad indicada como S5-A2 en el campus.

4. Síntesis del módulo

Un sistema operativo es un conjunto de programas que actúan como intermediarios entre el usuario y el hardware, facilitando la ejecución de aplicaciones y administrando los recursos del sistema. Se explica la relación entre el sistema operativo y el hardware, destacando la importancia del kernel o núcleo que se encarga de la gestión de recursos críticos como la CPU y la memoria.

Se explica la técnica de multiplexación por tiempo o espacio como forma de compartir en forma eficiente estos recursos.

Se abordan los objetivos y las funciones de los sistemas operativos.

Se vieron las principales clasificaciones abarcando diferentes criterios:

- Por su estructura: monolíticos, sistemas jerárquicos, microkernels, cliente/servidor, máquinas virtuales y exokernel.
- Por los servicios que ofrecen: monousuarios, multiusuarios, monotareas, multitareas, uniproceso y multiproceso (simétrico y asimétrico).
- Por la forma de entregar los servicios: sistemas operativos de red y distribuidos.
- Por interacción: sistemas por lotes (Batch), interactivos (Time Sharing) y sistemas de tiempo real (RTOS).

También se mencionan los tipos de sistemas operativos, una clasificación adicional, incluyendo aquellos diseñados para mainframes, servidores, computadoras personales, dispositivos embebidos, sensores y tarjetas inteligentes.

Con la ayuda de un material adicional publicado en el campus, se aborda la evolución histórica de los sistemas operativos para entender qué características tenían en sus inicios y cómo fueron evolucionando en el tiempo y su relación con el desarrollo de la industria del hardware.